

Dillon Mehta

Engineering Portfolio

Embedded systems, robotics, and multi-agent AI

I am an Electrical and Computer Engineering student at UT Austin. My work spans robot control software, custom electronics, mechanical design, and AI research. This portfolio presents three projects through the code, circuits, mechanisms, and experiments behind them.

dillonmehta@utexas.edu | [linkedin.com/in/dillon-mehta](https://www.linkedin.com/in/dillon-mehta) | github.com/DillonMehta | scholar.google.com/citations?user=tNUbaMQAAAAJ

Selected projects

Maze Navigation Robot

Pages 2-4

C++ motion control with encoder and gyro feedback, plus competition-specific route timing. I designed the PCB, soldered the assembly, and fabricated the shell. Includes code excerpts and reconstructed motor, controller, and sensor circuits.

VEX Climbing and Odometry

Page 5

A drivetrain-powered mechanism that reached the third rung of a four-foot ladder in under 12 seconds at Worlds. I designed the hang, pneumatic motor-sharing mechanism, and odometry hardware. Includes annotated hardware, CAD, and a competition photo.

Multi-Agent Medical AI Research

Pages 6-7

A study of how correct advice, misleading suggestions, and cognitive biases change AI diagnostic reasoning. As co-first and corresponding author, I directed the study, wrote the base simulation, and directed ablation programming. Includes methods, results, and a link to the paper.

Maze Navigation Robot

Firmware and motion control

C++ motion control, a custom PCB, and a fabricated chassis. The route and target time were configured separately for each competition.



I wrote the route parser, segment timing, motion profiles, encoder distance/speed calculations, and motor correction logic. I laid out the PCB in EAGLE, ordered it, soldered the assembly, and designed the printed shell in Fusion.

Across three hardware iterations, I added balloon tread for traction and a rear spoiler used as a counterweight. The robot earned seven medals, including second place at the MIT Invitational and Texas State Tournament.

The motion controller

For each straight segment, the controller ramps speed up for the first quarter, cruises for half, and ramps down for the final quarter. Encoder-derived velocity updates each PWM command; gyro angle corrects the right motor.

```
// Excerpts from fwd(); comments omitted and lines wrapped.
double delta_T = findTime(distance);
double delta_T_us = delta_T * 1e6;
if (elapsed_time <= delta_T_us / 4) {
    velocity_setpoint = (16.0 * distance) /
        (3.0 * delta_T * delta_T) * (elapsed_time / 1e6);
} else if (elapsed_time <= 3 * delta_T_us / 4) {
    velocity_setpoint = (4.0 * distance) / (3.0 * delta_T);
} else {
    double t_dec = elapsed_time - 3 * delta_T_us / 4;
    velocity_setpoint = (16.0 * distance) /
        (3.0 * delta_T * delta_T) *
        ((delta_T / 4) - t_dec / 1e6);
}
double velocity_error_L = velocity_setpoint - vL();
double velocity_error_R = velocity_setpoint - vR();
left_pwm += kP * velocity_error_L;
right_pwm += kP * velocity_error_R - kPs * ang();
left_pwm = constrain(left_pwm, pwm_min, 400);
right_pwm = constrain(right_pwm, pwm_min, 400);
motors.setSpeeds(left_pwm, right_pwm);
```

Timing, calibration, and termination

Route scheduling. `findTime(d)` reserves time for turns, then allocates straight-line time by distance: $(\text{targetTime} - \text{total_turns} * \text{turnTime} / 1e3) * (d / \text{total_distance})$. Both the command sequence and timing target change with the competition.

Feedback details. Distance uses 12 encoder counts per motor rotation, a 29.86 gearbox ratio, and separately calibrated wheel circumferences. Forward motion exits at the distance target or 100 ms beyond its allocated time. Turns stop on wheel travel while gyro angle adjusts motor effort.

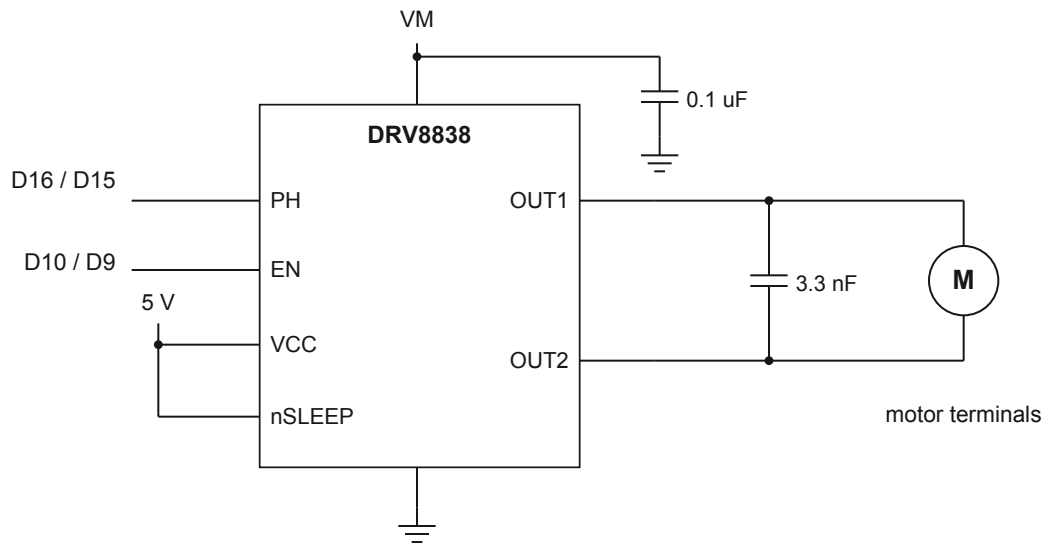
The circuit design continues on pages 3 and 4.

Maze Navigation Robot

Motor drive and encoder circuits

I designed these circuits and laid out the PCB in EAGLE, using Pololu schematics as a starting point. The diagrams below reconstruct the motor-drive and encoder sections.

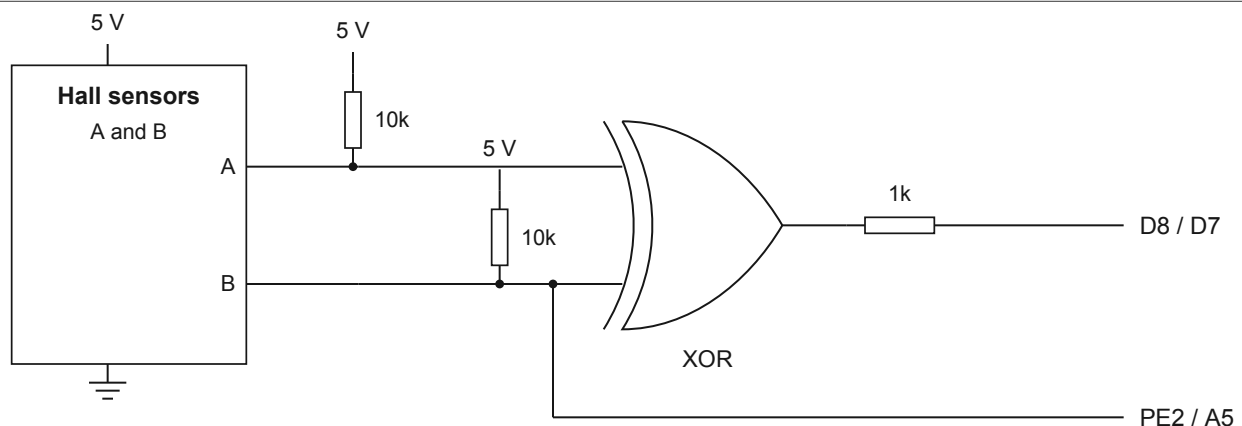
1. H-bridge motor drive - one channel shown



PH sets direction; EN receives PWM. Each motor has a 3.3 nF suppression capacitor, and each driver has local VM bypassing. Both channels share VM and ground.

Channel	PH / direction	EN / PWM	Encoder XOR	Encoder B
Left	D16	D10	D8	PE2
Right	D15	D9	D7	A5

2. Quadrature encoder conditioning - one wheel shown



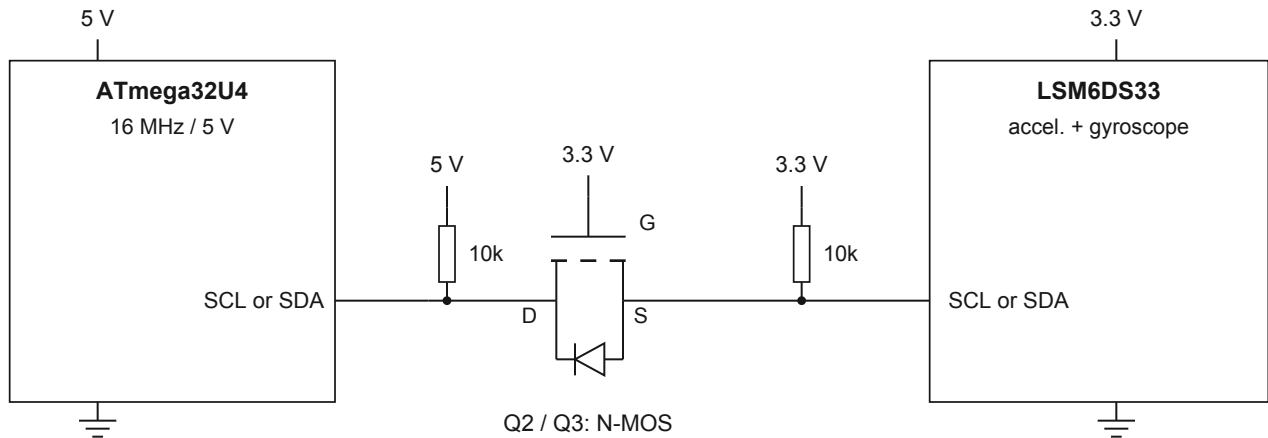
The XOR path and separate B channel provide the phase information used by the encoder library. The firmware converts counts to distance, then estimates wheel speed from distance change over elapsed time.

Maze Navigation Robot

Controller, IMU, and added sensors

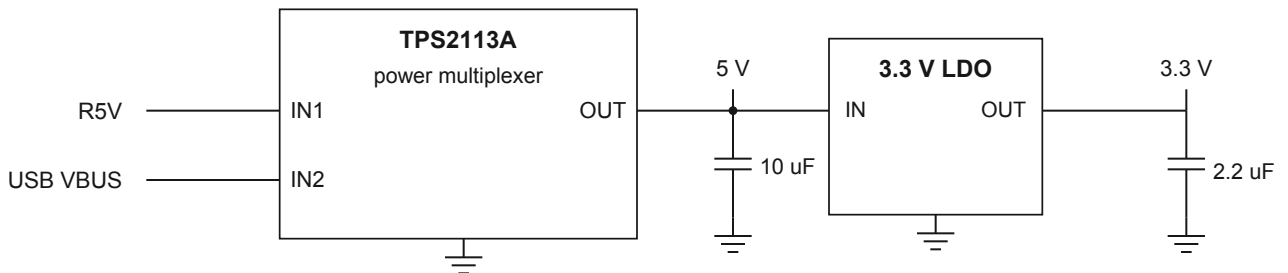
The controller and inertial sensor follow the base architecture. Bump sensing was removed. Bottom reflectance and a top-mounted IR sensor were added; the IMU was unchanged.

3. Controller and I2C inertial interface



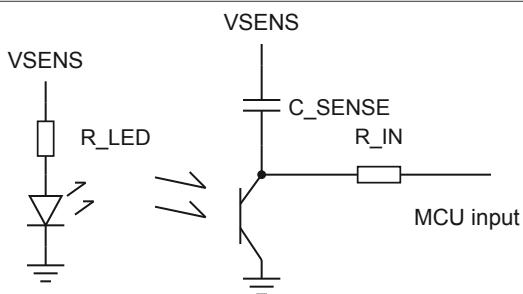
One channel is shown; the circuit repeats for SCL (D3) and SDA (D2). The 5 V controller communicates with the 3.3 V IMU through these two stages. The inherited gyro routine averages 1,024 stationary samples and integrates corrected Z-axis rate.

4. Logic power selection and sensor supply



R5V is the battery-derived regulated 5 V rail; VM separately supplies the motors. The multiplexer selects battery-derived logic power or USB. Local bypass capacitors and common ground complete the supply paths.

5. Added sensor interfaces



Bottom reflectance channel

Top IR: a separate top-mounted sensor connects to the controller. Its module, signal pin, and component values were not preserved. Added reflectance channel values are shown symbolically at left.

Circuit drawings reconstructed for this portfolio; the original PCB files are unavailable.

VEX Climbing and Odometry

Three-rung climb on a four-foot ladder

The VEX High Stakes endgame combined a ladder climb with a ring scored on the top stake. My robot reached the third rung in under 12 seconds at Worlds. Fewer than 5% of competitors could complete both tasks.

Active hooks

Actuated hooks engage the next rung and pull the robot upward.

Climb line

Routed string carries winch tension to the active hooks.

Drivetrain winch

The winch uses the existing drive motors to tension the climb line.



Passive hooks

Fixed hooks support the robot while the active hooks transition to the next rung.

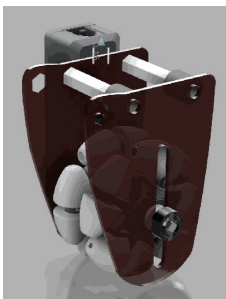
Power take-off

A pneumatic actuator shifts a gearbox from wheel drive to winch drive.

Drive-isolation feet

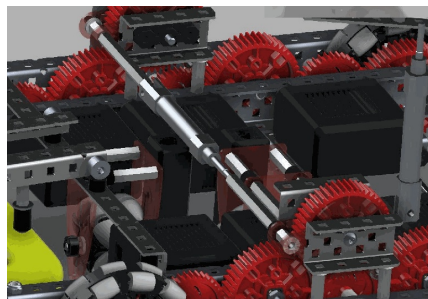
Passive feet lift the driven wheels off the floor for the initial climb.

CAD and the World Championship climb



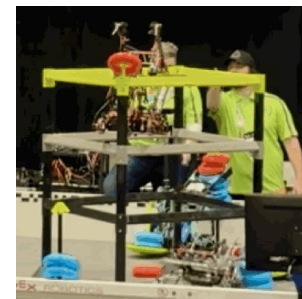
Odometry tracking pod

A separate tracking wheel and rotation sensor measure chassis motion.



Pneumatic power sharing

The actuator shifts drive power from the wheels to the climbing winch.



World Championship climb

Third-rung hang with the ring placed on the top stake.

I designed and integrated the mechanical systems; teammates wrote the VEX software.

Multi-Agent Medical AI Research

Simulation design and diagnostic interventions

This study asks when outside advice helps or misleads a multi-agent AI system making medical diagnoses. I tested correct guidance, incorrect suggestions, and nine cognitive biases at moments when the system's diagnosis changed.

Full paper: arxiv.org/abs/2609.02191

The simulation I built

As co-first and corresponding author, I led the study, built the GPT-4.1 simulation, and directed ablations. The code logs each turn's diagnosis. Published in CCIS; presented at NeurIPS workshops.

Agent	Responsibility in the experiment
Patient	Answers symptom/history questions during the first 10 turns.
Doctor	Requests information, records working diagnoses, selects a specialist, and makes the final diagnosis.
Specialist	Reviews the dialogue and discusses the case for up to five more turns.
Measurement	Returns case-grounded laboratory and imaging results when requested.
Priming	Generates the correct, incorrect, or bias-infused suggestion inserted before a Doctor turn.

How bias was injected. The Priming Agent combined the wrong diagnostic subcategory with a specific reasoning pattern: certainty without further tests for overconfidence, a recent similar case for availability, or prior effort as a reason to retain a diagnosis for sunk cost.

Detecting and replaying a fault point

I ran the 214-case MedQA set without intervention, embedded consecutive working diagnoses, and compared them with cosine similarity. Scores below the tenth percentile (0.5462) identified fault points; the lowest qualifying score selected the main intervention turn. Qualifying cases were rerun with the priming message at that location.

The configuration held temperature at 0.05 and limited outputs to 200 tokens. Evaluation combined Top-1/3/5 accuracy with diagnostic drift, requested tests, Doctor-Specialist disagreements, and early termination of the dialogue.

Correct and incorrect guidance produce different behavior

Intervention	Top-1	Top-3	Top-5
No intervention	50%	70%	74%
Correct subcategory	56%	72%	80%
Incorrect subcategory	48%	72%	78%
Correct + reasoning	60%	80%	86%
Incorrect + reasoning	48%	76%	80%

Correct guidance with reasoning raised Top-1 accuracy by 10 percentage points in this comparison. Incorrect cues lowered Top-1 accuracy even when the correct answer still appeared among the top five.

Multi-Agent Medical AI Research

Cognitive bias and intervention ablations

The bias experiments tested whether a misleading suggestion could redirect the system at a moment of diagnostic instability. Results below are reported for their own experiment groups.

For each of 50 assessed scenarios, the Priming Agent supplied an incorrect diagnostic suggestion with reasoning shaped by one of nine biases. This tested both the selected diagnosis and the alternatives the system continued to consider.

Bias condition	Top-1	Top-3	Top-5	Avg. diagnoses
No intervention	50%	76%	78%	8.75
Overconfidence	44%	72%	76%	9.68
Anchoring	50%	74%	78%	9.44
Availability	46%	70%	72%	9.02
Premature closure	48%	70%	74%	9.56
Representative heuristic	52%	74%	78%	9.02
Confirmation bias	48%	74%	80%	9.06
Omission bias	50%	72%	76%	8.84
Status quo	46%	66%	70%	9.10
Sunk cost	46%	70%	72%	9.16

What changed. Overconfidence had the largest Top-1 drop: 50% to 44%. Representative heuristics reached 52%, while anchoring and omission matched baseline. The effect depended on the cue; the experiment did not show every bias reducing accuracy.

Ablations: timing and repetition matter

Comparison	Design	Selected result
Phase of intervention	25 cases with faults in both patient and specialist phases.	Correct subcategory: 60% Top-1 in either phase alone, 76% across both. Baseline: 60%.
Repeated intervention	25 cases with three qualifying fault points.	Incorrect subcategory: 44.4% Top-1 with one intervention, 36% with three.
Fault definition	Compared consecutive-diagnosis drift with alignment to the correct answer.	Compared definitions on shared cases; ground-truth alignment selected a narrower case set.

Behavior and limits. Incorrect cues increased test requests and inter-agent disagreement. Correct cues sometimes shortened deliberation or bypassed specialist input. All agents shared one model, and prompt-injected biases on structured cases do not establish clinical behavior in real patients.